

Comic Craft

AI application designed to generate comic stories

PROJECT OVERVIEW

ComicCraft is an automated artificial intelligence application designed to generate comic stories. It addresses the complexity of comic creation by utilizing LLMs (Gemini Pro and Gemini Flash) for narrative generation and local diffusion models for panel image synthesis. The proposed system works by processing user requests through a web interface, generating the textual story, synthesizing corresponding images, assembling them into a structured comic layout, and exporting the final product as a PDF document.

Major technologies used include Python, Google Gemini API, Local Diffusion for image generation, and web technologies (HTML/CSS) for the interface.

TEAM MEMBERS

Team ID: SWTID-2026-4062

Class: II B.Sc COMPUTER SCIENCE

Department: COMPUTER SCIENCE

Shift: II

Name	Email	NM ID
Reena U	reenanithu777@gmail.com	92B9E17C3B8699FB5984CF2709D11ECA
Elakkiya A	elakkiya4100@gmail.com	44AA56F2F59C61AA1184020E8627D7CC
Jeevitha K	jeevithapraba535@gmail.com	2D3D42CD25DBDDC99774CB643C699379
Rithika S	rithu0267@gmail.com	63AA87CB8C54166BD3B4373E6BA31A5D
Mythili M	nn7963023@gmail.com	9EFA0254E9F92C7BBD44558A11CA0BDD

PHASE 1

BRAINSTORMING & IDEATION PHASE

The project provides an automated pipeline that integrates narrative generation (Gemini) with image synthesis (Local Diffusion) and layout formatting to instantly create exported PDF comic books from user prompts.

Objectives

- To automate the creation of comic book narratives using Gemini models.
- To generate contextually relevant comic panels using local diffusion image generation.
- To dynamically build comic layouts and export them to PDF format.

Expected Outcome

The expected practical outcome is a fully functional web-based tool capable of taking basic inputs and generating a complete, downloadable comic book PDF.

PHASE 2

REQUIREMENT ANALYSIS PHASE

1. Software Requirements

Based on the project structure, the following software components are required:

- **Programming Language:** Python
- **AI/LLM Integration:** Google Gemini Models (Gemini Pro, Gemini Flash)
- **Image Generation:** Local Diffusion models
- **Dependencies Management:** Managed via `requirements.txt` and `requirements-local-diffusion.txt`
- **Environment Configuration:** `.env` file for secure API key and environment variable storage.

3. Functional Requirements

- The system must generate story text utilizing Gemini Pro or Gemini Flash.
- The system must generate images corresponding to the story panels.
- The system must arrange text and images into a cohesive layout.
- The system must export the final layout as a PDF file.

PHASE 3

PROJECT DESIGN PHASE

Components Definition

Gemini Integrations (`gemini_pro.py`, `gemini_flash.py`)

- **Purpose:** To handle the narrative and text generation logic.
- **Responsibility:** Interacting with the Google Gemini API to generate story arcs, dialogue, and image generation prompts.

Image Generator (`image_generator.py`)

- **Purpose:** To synthesize visual panels.
- **Responsibility:** Interfacing with local diffusion models to turn text prompts into comic panel artwork.

Layout Builder (`layout_builder.py`)

- **Purpose:** To construct the visual flow of the comic.
- **Responsibility:** Combining generated text and images into structured pages and panels.

Exporters (**exporters.py**)

- **Purpose:** To format and save the final product.
- **Responsibility:** Converting the built layout into a downloadable PDF format (saved to **static/exports/**).

Web Application (**main.py**, **routes.py**, **schemas.py**)

- **Purpose:** To provide the user interface and handle application data.
- **Responsibility:** Managing HTTP requests/responses, validating data schemas, and serving static assets (CSS).

Module Design

The system is modularized within the **app/** directory, separating configuration (**config.py**), routing (**routes.py**), AI generation (**gemini_pro.py**, **image_generator.py**), and output formatting (**layout_builder.py**, **exporters.py**).

Security Design

- **Environment Variables:** Sensitive keys (like Gemini API keys) are stored securely in a **.env** file, preventing hardcoded credentials in the source code. The **.gitignore** file ensures these are not pushed to version control.

PHASE 4

PROJECT PLANNING PHASE

Project Modules

- Configuration Module
- Routing/Web Application Module
- Data Validation (Schemas) Module
- LLM Narrative Module (Gemini Pro/Flash)
- Image Synthesis Module (Local Diffusion)
- Layout Construction Module
- PDF Export Module

Key Features

- **Dual-Model LLM Support:** Utilizes both Gemini Pro and Gemini Flash for optimized text generation tasks.
- **Local Image Generation:** Employs diffusion models locally to generate panel art without relying on external paid image APIs.
- **Automated PDF Export:** Dynamically saves the assembled comic to the `static/exports/` directory.

PHASE 5

PROJECT DEVELOPMENT PHASE

Development Environment Setup

The environment requires a Python installation, configuration of environment variables via the `.env` file, and installation of packages defined in `requirements.txt` and `requirements-local-diffusion.txt`.

Project Folder Structure

```
ComicCraft_AI/
├── .env
├── .env.example
├── .gitignore
├── requirements-local-diffusion.txt
├── requirements.txt
├── ComicCraft - AI Comic Story Creator using Gemini Models.pdf
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── exporters.py
│   ├── gemini_flash.py
│   ├── gemini_pro.py
│   ├── image_generator.py
│   ├── layout_builder.py
│   ├── main.py
│   ├── routes.py
│   └── schemas.py
├── static/
│   ├── css/
│   │   └── style.css
│   └── exports/
│       └── comic_20260930_153431_523899.pdf
```

Installation and Configuration

- **Dependencies:** Python packages must be installed using `pip install -r requirements.txt` and `pip install -r requirements-local-diffusion.txt`.
- **Configuration:** The `.env.example` file must be copied to `.env` and populated with the necessary API keys and local configuration paths.

Module Implementation

Module 1: Application Core

Implemented via `app/main.py` and `app/routes.py`, establishing the web server and API/frontend endpoints. Data structures are validated using `app/schemas.py`. Configuration logic is housed in `app/config.py`.

Module 2: AI Generation

Implemented via `app/gemini_pro.py`, `app/gemini_flash.py`, and `app/image_generator.py`. The Gemini scripts handle prompt engineering and API calls to Google's models, while the image generator interfaces with the local diffusion setup.

Module 3: Assembly & Export

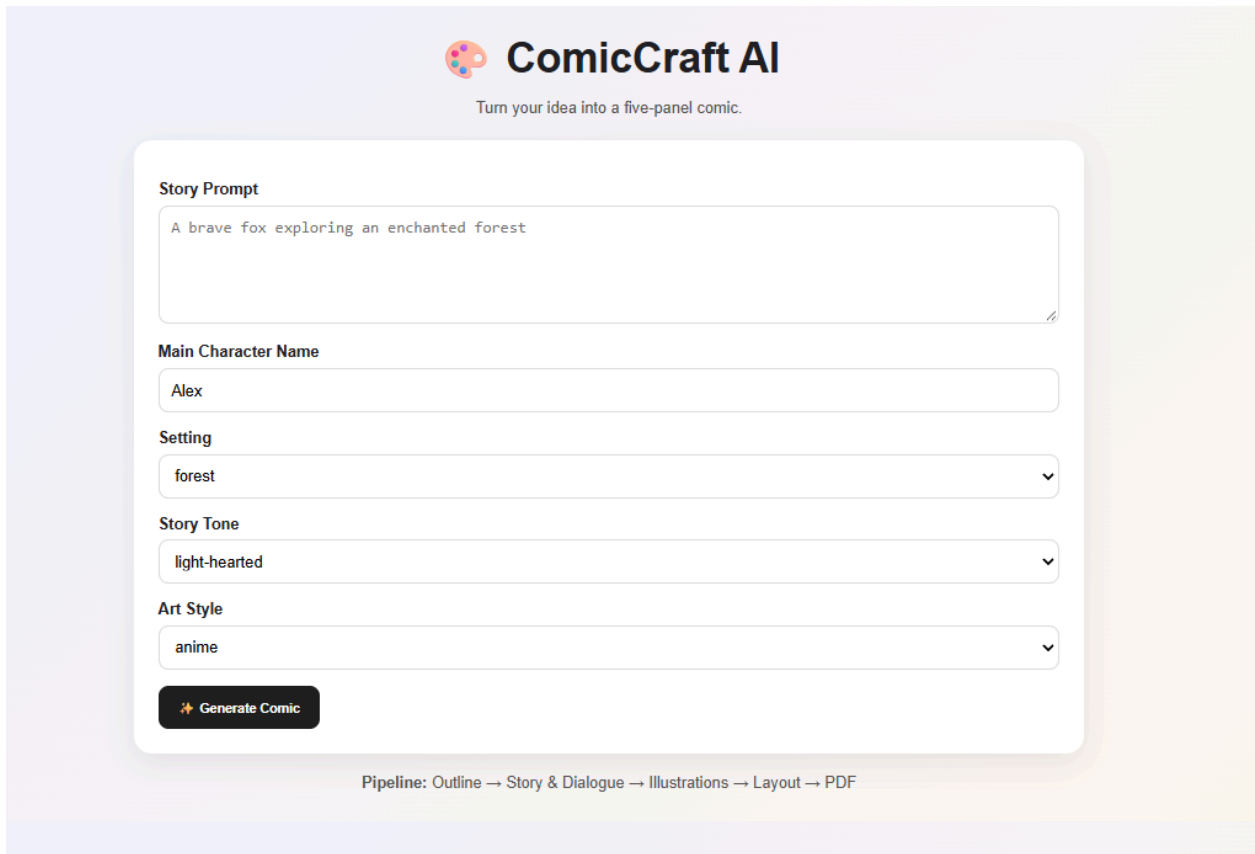
Implemented via `app/layout_builder.py` and `app/exporters.py`. The layout builder aligns the generated text and images, passing the structured data to the exporter, which writes the binary PDF file to the `static/exports/` directory.

Core Functionality

The core pipeline takes a user prompt, routes it to the Gemini models to generate a script and image prompts, passes the image prompts to the local diffusion model, and finally pipes all assets into the layout builder and exporter to render the PDF.

User Interface Implementation

The project features a web-based user interface, styled using CSS located at [static/css/style.css](#).



ComicCraft AI
Turn your idea into a five-panel comic.

Story Prompt
A brave fox exploring an enchanted forest

Main Character Name
Alex

Setting
forest

Story Tone
light-hearted

Art Style
anime

Generate Comic

Pipeline: Outline → Story & Dialogue → Illustrations → Layout → PDF

Figure: Web Interface

Integration

- **Backend and AI Services:** Direct API integration with Google Gemini Pro and Flash.
- **Backend and Image Generation:** Integration with a local diffusion model environment.

GitHub Repository: [REPO LINK](#)

PHASE 6

PROJECT TESTING PHASE

Testing Objectives

- To verify that the narrative generation utilizing Gemini models (`gemini_pro.py` and `gemini_flash.py`) functions correctly based on user input.
- To ensure the `image_generator.py` successfully synthesizes corresponding panel artwork using local diffusion models.
- To confirm that the `layout_builder.py` correctly assembles text and images into cohesive, structured comic pages.
- To validate that the `exporters.py` module successfully formats and saves the final layout as a PDF document into the `static/exports/` directory.
- To ensure secure loading of API keys and configurations via the `.env` file and `config.py`.

Testing Environment

- **Backend Framework:** Python environment configured with dependencies from `requirements.txt` and `requirements-local-diffusion.txt`.
- **Frontend:** Web browsers interacting with the HTML/CSS interface via `main.py` and `routes.py`.
- **Security/Config:** Local environment containing the `.env` file for secure API key and path management.

Testing Methods

- **Unit Testing:** Isolating and validating individual modules, specifically the text generator, image generator, and layout builder.
- **Integration Testing:** Verifying the data flow between the Gemini API (text/prompts) and the local diffusion environment (images) into the final layout structurer.
- **System/End-to-End Testing:** Running the complete pipeline from web interface prompt submission to final PDF download.

Final Testing Result The ComicCraft application is stable, secure, and functionally complete according to the initial project requirements. The system is fully prepared for the Phase 8 demonstration.

PHASE 7

PROJECT DOCUMENTATION PHASE

Project Introduction

ComicCraft is an automated AI comic story creator that leverages Google Gemini models and local diffusion technology to synthesize narratives and panel art.

Objectives

To provide an automated pipeline for generating, formatting, and exporting AI-driven comic books based on user input.

Technologies Used

Python, Google Gemini Pro, Google Gemini Flash, Local Diffusion, HTML/CSS, PDF Generation Libraries.

System Requirements

Requires Python, requisite pip packages (including local diffusion dependencies), and valid API keys stored in a `.env` file.

System Design

The system uses a modular architecture separating web routing, AI text generation, image synthesis, layout construction, and file exporting.

Implementation

Implemented via a Python backend using modular scripts (`gemini_pro.py`, `image_generator.py`, `layout_builder.py`, `exporters.py`) orchestrated by a central web application (`main.py`, `routes.py`).

Features

- Automated story generation via Gemini.
- Automated comic panel art generation via Local Diffusion.
- Automated PDF compilation and export.

Screenshots

 **ComicCraft AI**

Turn your idea into a five-panel comic.

Story Prompt

A brave fox exploring an enchanted forest

Main Character Name

Alex

Setting

forest

Story Tone

light-hearted

Art Style

anime

Generate Comic

Pipeline: Outline → Story & Dialogue → Illustrations → Layout → PDF

 **Your Comic**

Download Your Comic as PDF

Panel 1: The Beginning



Alex begins the adventure in the school.

Caption: The Beginning — Poetic moment.

Narration: Alex faces the moment described in this panel. Alex begins the adventure in the school.

► Image Prompt Reference

Panel 5: A New Beginning



Alex reaches the end of this adventure with a memorable lesson.

Caption: A New Beginning — Poetic moment.

Narration: Alex faces the moment described in this panel. Alex reaches the end of this adventure with a memorable lesson.

▼ Image Prompt Reference

Alex celebrating at the end of an adventure in a school, anime comic art, warm ending, cinematic

Create Another Comic

Advantages

- End-to-end automation of comic creation.
- Utilizes advanced LLMs for coherent storytelling.
- Cost-effective local image generation.

Conclusion

ComicCraft successfully demonstrates the integration of multiple distinct AI models—combining the textual reasoning capabilities of Google Gemini with the visual synthesis of local diffusion models—into a single, cohesive software pipeline capable of producing formatted digital media.

GitHub Repository: [REPO LINK](#)

PHASE 8

PROJECT DEMONSTRATION PHASE

Project Introduction

- **Project Name:** ComicCraft - AI Comic Story Creator using Gemini Models
- **Purpose of the Project:** To automate the creation and layout of comic stories using generative AI.

Project Overview

The demonstration will showcase the application receiving a prompt, generating the story via Gemini, synthesizing images via local diffusion, and exporting the final PDF.

Project File / Code Structure

The demonstration will highlight the `app/` directory, specifically showing how `routes.py` connects to `gemini_pro.py` and `image_generator.py`, and how the final output is routed to `static/exports/`.

Project Execution

Demonstrate starting the web server via `main.py` and accessing the UI.

Feature Demonstration

- **Story Generation:** Showcasing the script generated by Gemini.
- **Image Synthesis:** Showcasing the panels generated by the local diffusion model.
- **PDF Export:** Showcasing the final document generated by `exporters.py`.

Complete User Workflow

The video will trace the user entering a prompt, waiting for the generation pipeline to complete, and downloading the final PDF.

Final Output

Opening the generated file (e.g., [comic_20260930_153431_523899.pdf](#)) to demonstrate the final assembled comic.

Benefits / Use of the Project

Illustrating how creators, hobbyists, or writers can rapidly prototype visual stories.

Conclusion

A brief summary of the successful execution of the ComicCraft AI pipeline.

Demo Video: [Demo Link](#)